SQA_FinalPresentationAndReport.docx

AUT University

Document Details

Submission ID
trn:oid:::20437:155034950

Submission Date
Oct 18, 2026, 2:52 PM GMT+13

Download Date
Oct 18, 2026, 2:59 PM GMT+13

File Name
SQA_FinalPresentationAndReport.txt

File Size
50.5 KB

20 Pages

7,819 Words

43,535 Characters

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

17 Not Cited or Quoted 4%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
2% Publications
4% Submitted works (Student Papers)

Match Groups

- **17** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
2% Publications
4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	University of Portsmouth on 2026-05-20 <1%
2	Submitted works	De La Salle University - Manila on 2026-09-02 <1%
3	Submitted works	Asia Pacific International College on 2026-07-20 <1%
4	Internet	dokumen.pub <1%
5	Publication	Nimley, Kenneth Boyenah Dube Boute. "Aligning Information Systems With Com... <1%
6	Submitted works	Uniminuto Virtual on 2026-10-17 <1%
7	Submitted works	Johns Hopkins Unversity on 2026-08-12 <1%
8	Submitted works	Monash University on 2026-10-14 <1%
9	Submitted works	Brunel University on 2026-04-10 <1%
10	Internet	ipfs.io <1%

11 Internet

www.ling.lu.se <1%

12 Submitted works

University of Pretoria on 2026-10-06 <1%

13 Internet

hdl.handle.net <1%

14 Submitted works

Blue Mountain Hotel School on 2025-07-27 <1%

15 Submitted works

Polytechnic Institute Australia on 2023-12-17 <1%

Task 1: Project Completion and Final Implementation
Requirements refinement (not scope change)
NFR-09's cold-start threshold was changed from a placeholder of <2000ms to <500ms once real data showed the original threshold was too lenient. NFR-02's threshold (>= 15FPS for 95% of frames) was implemented and tested against real data, revealing the system always achieves ~13FPS under baseline CPU-only inference, which is an NFR failure, not an implementation gap. This brings the question of whether 15FPS is a realistic target for CPU-bound embedded-simulated inference, which we will evaluate in Task 8, with the review in Task 10, rather than treat it as a defect to fix by lowering the threshold.

Implementation additions (Gaps in Assessment 1)
NFR-03 (memory usage) previously had a defined threshold with no implemented measurement; this has been implemented using process working set memory sampling and is now actively tested

NFR-08 (error handling) previously only verified that no unhandled exception occurred during a run; this has been changed with four dedicated test cases validating each specific error condition individually (missing model, corrupted frame, invalid input path), addressing the "false pass" seen in Assessment 1 feedback. Under ErrorHandlingTests are five tests, covering Missing model (FR-09), Missing/invalid input path (FR-11), Single corrupted frame isolated (FR-10), and corrupted frame within a batch, confirming continuation (FR-10).

NFR-04, NFR-06, and NFR-07 previously untested, now have dedicated automated test coverage confirming reliability under load, consistency across repeated runs, and portability across hardware profiles.
NFR-04

NFR-06

NFR-07

Architectural or technological changes
The core architecture (GUI (DetectionPipelineRunner (Detector/ResourceConstraintSimulator/MetricsLogging (SummaryReportGenerator) and the stack (C#/.NET 9, MAUI Windows, YoloDotNet/ONNX Runtime, MSTest) are unchanged from Assessment 1. Development since then is focused on finishing test coverage and

measurement gaps seen in feedback, not restructuring the system.
Architecture/Data Flow diagram

GUI(Profile: Baseline, Load condition: Baseline)
GUI(Profile: Baseline, Load condition: High load)
GUI(Profile: Constrained, Load condition: Baseline)
GUI(Profile: Constrained, Load condition: High load)
Deferred, not implemented
A feature which would allow users to upload their own dataset for testing was evaluated and deliberated on by Yaacoub and Matthew, we concluded that it doesn't map to any required Assessment 2 task and was judged not a good use of our time. This is documented as a future recommendation (Task 10) rather than abandoned.
NFR-05 (configurability) hasn't been automated, since it's fundamentally a GUI usability requirement rather than backend logic suited to unit testing. It'll be validated as a manual usability test case under Task 7 (Quality Testing), supported by screenshots.
Task 2: Requirements Verification and Traceability
Review and finalisation of requirements
The baseline is made up of 11 functional requirements (FR-01 to FR-11) and 9 non-functional requirements (NFR-01 to NFR-09), which have remained constant since Assessment 1, with no additions or deletions made. Only the testing of these requirements has changed; NFR-03 is checked by a real memory measurement, NFR-08 by a different test for each error condition, and NFR-02, NFR-04, NFR-06, and NFR-07 are now tested with automated tests, whereas at mid-point, none of these four had been tested.
The numbering of test cases here replaces the nine-test case numbering used in the traceability matrix for Assessment 1. The 15FPS mark for NFR-02 stands, with the same failure in the current system; this is evaluated in Task 8, with the review in Task 10, rather than reducing the bar.
Comparison of the baseline with the final system reveals three discrepancies between the code and the requirements.
NFR-01 and NFR-02 are functionally overlapping, because at 15FPS (a frame time of 66.7ms) any frame satisfying NFR-02 will also satisfy NFR-01's 100ms barrier. This is, however, not remedied, kept as originally written in Assessment 1.

Requirements Traceability Matrix
Test case IDs refer to the final test case table in Task 3. Defect IDs refer to the table that follows and are to be aligned with the defect register in Task 6.

ID	Requirement	Implemented in	Test case(s)	Result	Defect
FR-01	Load and run a pre-trained detection model on input data	OnnxYoloDetector (constructor)	TC-01	Pass	None
FR-02	Detect and classify objects in each frame	OnnxYoloDetector.DetectFrame	TC-01, TC-03	Partial (accuracy unverified)	None
FR-03	Output bounding boxes, labels and confidence scores	DetectionResult	TC-01, TC-03	Pass	None
FR-04	Process pre-recorded input as a simulated feed	DetectionPipelineRunner.ProcessBatch	TC-01, TC-04,	Pass	None
FR-05	Log each frame's results and timestamp	MetricsLogger, FrameResult, SaveToJson	TC-04, TC-05	Pass	None
FR-06					

Configurable simulated resource constraint before a run
ResourceConstraintProfile, ResourceConstraintSimulator, GUI profile picker TC-06, TC-07 Partial (no memory cap) None (scope limitation) FR-07 Summary report of accuracy and performance metrics
SummaryReportGenerator, GUI results panel TC-08, TC-09, TC-10
Partial (no accuracy metric) DEF-05 (closed) FR-08 Select between at least two load conditions LoadCondition, ProcessBatch, GUI load picker TC-11 Pass None FR-09 Stop and log an error if the model cannot loadOnnxYoloDetector (constructor) TC-12 Pass None
FR-10 Skip a corrupted frame, log it, continue
DetectionPipelineRunner.ProcessFrames TC-14 Pass None FR-11 Reject a run before execution if input is missing or invalid
DetectionPipelineRunner.ValidateInputPaths TC-13, TC-15 Pass
None NFR-01 Frame processing time under 100ms
SummaryReportGenerator (NFR-01 check) TC-06, TC-08, TC-16 Pass for Baseline/Baseline (78.3ms); exceeded in the other three (381.6,1773.0,6078.9ms) None NFR-02 At least 95% of frames sustain 15FPS SummaryReportGenerator (NFR-02 check) TC-09, TC-16 Fail (0.0% of frames; about 13FPS average) DEF-01 NFR-03 Peak memory under 512MB FrameResult.MemoryUsedBytes, SummaryReportGenerator (NFR-03 check) TC-16 Pass (about 180 to 238MB in TC-16) DEF-03 (closed) NFR-04 100% of frames complete under high load SummaryReportGenerator (NFR-04 check), Parallel.For in ProcessBatch TC-16 Pass (10/10 completed in all four configurations)
None NFR-05 Configure and launch a run in 3 steps or fewer, without editing source MainPage.xaml (two pickers and a run button) Manual GUI test (Task 7) Not automated DEF-07, DEF-08
NFR-06 Results within 2% variance across 5 repeated runs
DetectionTests variance check TC-02 Partial (0% variance, detection counts only) None NFR-07 Full run completes under at least 2 simulated hardware profiles ResourceConstraintSimulator, Baseline and Constrained profiles TC-07, TC-16 Pass None NFR-08 All defined error conditions handled without crashing
ProcessFrames try/catch, ValidateInputPaths, detector constructor, NFR-08 report check TC-12, TC-13, TC-14, TC-15 Pass DEF-04, DEF-05 (closed) NFR-09 Cold-start latency under 500ms
SummaryReportGenerator (NFR-09 check) TC-10 Pass None
Defect Mapping
ID
Defect or findingRelated requirement and test Status DEF-01
System sustains about 13FPS on CPU-only YOLOv8n inference against a 15FPS threshold. NFR-02, TC-09 Open. Threshold review in Task 8.
DEF-02 Parallel test execution leaked the CPU affinity setting between tests, producing inconsistent latency results. NFR-01, NFR-02
Closed. DoNotParallelize applied and affinity reset in test setup and cleanup. DEF-03 NFR-03 had a defined threshold but no measurement in code. NFR-03 Closed. Peak working set sampled per frame and checked in the report. DEF-04 NFR-08 report check passed unconditionally if the run returned. NFR-08, TC-12 to TC-15
Closed. Per-condition tests added. The report check remains a runtime check only. DEF-05 The NFR-08 description string shown in the GUI and test output cites TC-07, TC-09 and TC-10, which do not match the final test case numbering (TC-12 to TC-15).FR-07, NFR-08
Open. Low severity. DEF-06 MSTestSettings.cs requests

method-level parallel execution while AssemblyInfo.cs sets DoNotParallelize. The effective behaviour has not been confirmed. NFR-01, NFR-02 (timing reliability) Open. Low severity. FR-06 (partial). The constrained profile pins the process to one CPU core and adds 50ms of latency per frame. No memory cap exists; memory is measured against NFR-03 but not enforced. This is enough to demonstrate NFR-07. FR-07 (partial) and accuracy (unverified). The report gives latency, memory, detection counts and NFR pass/fail but no accuracy measure, and no comparison against labelled samples was done. TC-01 and TC-03 only check that detections are produced. Model accuracy is outside this project's scope.

NFR-02 (implemented, not passing). TC-09 tests the requirement correctly and fails for all runs (0.0% of frames greater than or equal to 15FPS). The test itself is passing in MSTest since the test only asserts the number of frames, so the failure appears in the report output and not as a red test in Test Explorer. Recorded as DEF-01 and taken to Task 8, where the realism of 15FPS for CPU-only inference is considered.

NFR-01 (validated for Baseline/Baseline only). Met at 78.3ms and exceeded in the other three configurations. The requirement says each frame under 100ms, the acceptance criterion says 95% of frames, and the implementation checks the average.

NFR-05 (verified manually). A GUI usability requirement, tested by the manual cases in Task 7.

NFR-06 (partial). TC-02 checks detection counts only, not bounding-box variance

Sample data coverage (limitation). All tests employ two base images, repeated in volume. Inclusion of an option for users to provide their own data set was initially slated for a future Task 10 recommendation, as this functionality does not appear to translate into a required Assessment 2 task.

Security, accessibility and compatibility (out of scope). No requirements were defined; the system is local-only with one dashboard page and one platform (.NET MAUI on Windows).

Task 3: Test Execution and Final Test Results

Execution Summary

Metric	Count				
Planned	16	Executed	16		
Passed	14	Failed	2 (TC-09, TC-16)		
Blocked	0	Not Run	0		

Explanation of Failed Test and Corrective Action

TC-09 and TC-16 fail because the system misses NFR thresholds, not because the tests are wrong: both methods run correctly. TC-09 shows ~13FPS on a 20-frame sample against 15FPS. In TC-16, NFR-01 is met only under Baseline/Baseline and NFR-02 fails in every combination. These are recorded as NFR non-conformances, not code defects, since they show the project's main finding: functional correctness does not guarantee real-time compliance. The corrective action is at requirements level: whether 15FPS is realistic for CPU-only inference, and how NFR-01 applies under constrained and high-load conditions, is evaluated in Task 8 with the review in Task 10. Neither result is a regression.

Status Convention

Status reflects whether the system met the test's expected result, NFR thresholds included. TC-09 and TC-16 are marked Fail because the system does not meet NFR-01 and NFR-02, even though their MSTests pass and CI reports all 16 methods passing.

Test Case Table

ID Test Case Preconditions Steps Input Expected Result Requirements Type Priority Status TC-01 Detector returns valid detections on known sample image Model + sample image available 1. Load detector 2. Call DetectFrame(bus.jpg) bus.jpg Returns list of detections with labels, confidence, bounding box FR-01-04 Unit/Integration High Pass TC-02 Detection results consistent across repeated runs Model loaded, warmed up 1. Run DetectFrame 5x on same image bus.jpg x5 Detection count variance ≤2% NFR-06 Non-functional Medium Pass (0% variance) TC-03 Detector returns empty list on blank image Model loaded 1. Generate blank image 2. Run DetectFrame Blank white imageReturns empty detection list, no error FR-02, FR-03 Boundary Medium Pass TC-04 Full pipeline logs results and timing for multiple frames Model + 2 sample images available 1. Process bus.jpg, frame 1 2. Process zidane.jpg, frame 2 bus.jpg, zidane.jpg 2 FrameResults logged with valid detections, timestamps, latency FR-04, FR-05 Integration High Pass TC-05 Logged results export to valid, parsable JSON Frames processed and logged 1. Call SaveToJson() 2. Deserialize output Logged FrameResult list File exists, deserializes without error, matches frame count FR-05 Integration Medium Pass TC-06 Constrained resource profile produces higher latency than baseline Model loaded 1. Run under Baseline profile 2. Run under Constrained profile bus.jpg, both profiles Constrained latency measurably higher FR-06, NFR-01 Non-functional High Pass TC-07 System completes a full run under both resource profiles Model + sample images available 1. Run batch under Baseline 2. Run batch under Constrained bus.jpg, zidane.jpg Both runs complete, 0 frames skipped FR-06, NFR-07 Non-functional High Pass TC-08 Summary report evaluates latency (NFR-01) pass/fail against thresholdFrames processed 1. Generate summary report Logged FrameResults NFR-01 result reflects accurate pass/fail vs 100ms threshold FR-07, NFR-01 Non-functional High Pass TC-09 Frame rate compliance evaluated across larger batch Model warmed up 1. Process 20-frame batch 2. Generate report 20 images (repeated samples) NFR-02 result reflects ≥95% of frames sustain ≥15FPS FR-07, NFR-02 Non-functional Medium Pass TC-10 Cold-start latency measured separately from steady-state Fresh detector instance 1. Measure first DetectFrame call 2. Process further frames bus.jpg Cold-start and steady-state latency reported separately FR-07, NFR-09 Non-functional Medium Pass TC-11 Batch processing completes under Baseline and HighLoad conditions Model + sample images available 1. Run batch, Baseline load 2. Run batch, HighLoad 4 images Both complete all frames, 0 skipped FR-08 Functional High Pass TC-12 Detector constructor throws clear error on missing model No model file at given path 1. Construct OnnxYoloDetector with invalid path Invalid model path FileNotFoundException thrown with clear message FR-09, NFR-08 Negative High Pass TC-13 Batch run rejected before execution on missing input file Model available 1. Call ProcessBatch with non-existent image path Invalid image path FileNotFoundException

thrown, no frames processed FR-11, NFR-08 Negative High Pass
TC-14 Corrupted frame is skipped and logged, run continues
Model available, 1 corrupted + 1 valid file 1. Process corrupted file alone
2. Process batch with corrupted + valid file Corrupted file, bus.jpg
Corrupted frame marked skipped with reason; valid frame still processed FR-10, NFR-08 Negative High Pass TC-15 Empty batch request is rejected before execution Model available 1. Call ProcessBatch with empty list Empty list ArgumentException thrown FR-11, NFR-08 Boundary/NegativeMedium Pass TC-16 Performance across all resource-profile and load-condition combinations Model and both sample images available; CPU affinity reset at start; a warm-up frame run under each profile 1. For each of Baseline/Constrained x Baseline/HighLoad, apply the profile
2. Run a warm-up frame, then process a 10-frame batch under the load condition
3. Generate the summary report and record latency, throughput and NFR results 10 frames per combination, alternating bus.jpg and zidane.jpg (40 recorded frames) All 4 combinations complete 10/10 frames with 0 skipped; NFR-01 average latency <100ms; NFR-02 ≥95% of frames at ≥15FPS; NFR-03 peak memory <512MB NFR-01-04, NFR-07Non-functional (performance/reliability) High Fail: completion, NFR-03, NFR-04 and NFR-07 met; NFR-01 failed in 3 of 4 combinations (only Baseline/Baseline passed, 78.3ms) and NFR-02 failed in all 4 (Baseline/Baseline mean 12.8FPS per frame, best throughput 15.9FPS)

Regression Testing
The full MSTest suite is re-run after every code change via Visual Studio's Test Explorer, following the regression practice after the Assessment 1 concurrency bug (Parallel test execution caused leakage between tests). [assembly: DoNotParallelize] stays to guarantee repeatable results across runs, and all 16 test cases are re-verified as part of this regression process before each commit.
Remaining Quality Risks
The NFR-02 failures (TC-09, TC-16) and the NFR-01 failures under Constrained and HighLoad conditions (TC-16) remain open and feed the Task 8 release decision.
NFR-05 (configurability) is validated manually rather than through automated testing (See Task 7), which holds a risk of being under-documented compared to the automated suite.
Test coverage relies on a small, fixed sample image set (2 base images, repeated for volume), which lowers confidence in detection-accuracy claims in real-world conditions.
Task 4: Test Automation and Quality Engineering
Selection of test cases for automation
Test cases were selected for automation when they had a known input, an expected result, and a cost to repeat by hand that was higher than the cost of writing the test. By these criteria all 16 test cases in Task 3 were automated. The tests fall into five groups.
Group Test cases Justification Detection Correctness TC-01, TC-03 A fixed sample image and a blank image give expected outputs, so the result is the same on every run. Pipeline and logging TC-04, TC-05 These check the full path from frame to logged result and JSON export, which must be re-verified after every code change. Resource profiles and load conditions TC-06, TC-07, TC-11, TC-16 The requirements need runs under two profiles and two load conditions. Doing this by hand for every change would be slow and inconsistent. NFR measurement TC-02, TC-08 to TC-10 Threshold for variance, latency, frame rate and cold start are calculated in code, so a script is more accurate than a person reading a screen. Error Handling TC-12 to TC-15 A missing model, a corrupted frame, a missing input file and an empty batch are hard to trigger consistently by hand and cheap to automate. Implementation
Tests written in MSTest are run through the Visual Studio Test Explorer. These consist of a set of 16 tests split across 7 classes: DetectionTests (3), ErrorHandlingTests (5), LoadConditionTests (1), PipelineRunnerTests (2), ResourceConstraintTests (2), SummaryReportTests (2) and PerformanceMatrixTests (1). These run the YOLOv8n model on images and do not use mocks, ensuring that the same code path is executed as in the GUI. In addition, the thresholds are evaluated by the summary report generator, so the GUI and the automated test suite use the same set of non-functional requirement validation checks. The tests reset CPU affinity in TestInitialize and TestCleanup and use the DoNotParallelize attribute (added as the fix for DEF-02 in Task 6). Finally, the test suite is run on every push to main and every pull request into main via the GitHub Actions workflow discussed in Task 5, with .trx results uploaded as artifacts.
Execution results
The execution and completion of all 16 test methods occur normally, with passing results in MSTest and inside the CI environment. Task 3 records

that 14 test cases passed, while 2 (TC-09 and TC-16) failed. This is because the status in Task 3 reflects whether the NFR thresholds were satisfied. The outputs of TC-09 and TC-16 report the thresholds as not met, however this is not asserted; the reasoning for this is that absolute timings are machine specific and hence unreliable for CI, so the failure appears in the report output rather than as red tests inside Test Explorer.

12 Test Explorer results for NFR-02 to NFR-04, and NFR-06 to NFR-08 are shown in Task 1, and the full set of screenshots is in the root of the repository, with file names stating NFR-02, NFR-03 and so on. CI evidence is in Task 5.

Benefits

The automated suite can re-run the complete pipeline within minutes of every change to it. In addition, the whole suite is run by CI after every push to the main branch and every pull request into the main branch. It evaluates the NFR thresholds consistently on every run and found real issues such as the 15FPS gap (DEF-01) and the affinity leakage issue during execution of parallel tests (DEF-02), and it guards against a false NFR-08 pass (DEF-04).

Limitations

Timing Sensitivity

A host's load affects timing, so a result close to a threshold could pass or fail depending on what else the machine is running. Timing thresholds are therefore reported, not asserted.

Failures can be hidden in the test result

TC-09 and TC-16 show as passed in Test Explorer even though their NFR thresholds aren't met, because thresholds are reported, not asserted.

Input set

The tests only use two sample images, meaning detection accuracy is not verified, and bounding box variance is not being tested either.

Coverage/Mocking

Code coverage was not measured, and coverage thresholds were not defined. Furthermore, there is no use of mocks via IDetector, meaning pipeline logic cannot be tested without the presence of the model file.

Maintainability

The tests are structured around the primary areas of interest with each class relating to the appropriate area of the system. Test names are descriptive of the behaviour which is verified. Threshold values for NFRs are centralized in the summary report generator rather than embedded in each test so changing a threshold shouldn't require code changes to the tests. Nonetheless, maintainability is reduced by one thing:The affinity reset function is copied into 5 of the 7 test classes and should move into a shared base class.

Where automation is not appropriate

The usability requirement NFR-05 (3 steps to configure and execute) applies to the user experience when using the GUI. Therefore, this has been verified manually via screenshots and the use of independent testers (Task 7).

Performing exploratory testing upon the GUI has raised issues that are unlikely to have been caught by the scripted testing. Notably: a run with no selection that defaults to Baseline/Baseline without a message (DEF-07) and option labels which do not fully describe what they do (DEF-08).

Automating the detection accuracy check was not done because it would require the construction of a labelled data set, which was outside the

scope of this project (Task 2).
Task 5: Continuous Integration and DevOps QA
Development and Integration Workflow
Development follows a feature-branch workflow: each team member works on a specific branch for their assigned tasks, committing regularly before merging into main via GitHub at the end. This branch-per-task structure keeps individual contributions traceable and reviewable, which supports the GitHub-evidence requirement for the contribution statement (Task 11). CI was implemented and runs when changes are pushed to main or a pull request into main is opened, so work on feature branches is checked at merge rather than on every individual commit.
CI Workflow Implementation
A GitHub Actions workflow was made to automatically build and test the EmbeddedCV.Tests project on a Windows runner on every push and pull request to main and can also be triggered manually. The MAUI GUI project is excluded because it needs the Windows desktop workload and the tests don’t depend on it. This returns automated feedback on changes that break the build or test suite.
(See .github/workflows/ci.yml for the file)
CI Build test:

Run Tests Step:

Proof of the CI Action in Github:
Quality Gates Defined
Successful build: The workflow fails if the restore or build step fails, flagging broken code.
All tests passing: The workflow fails if any of the 16 tests fail. NFR threshold results are reported in the test output but don’t gate the build. Many depend on hardware speed and would make CI unreliable on GitHub’s runners.
Test result retention: .trx results are uploaded as artifacts every run, giving a reviewable record of which tests passed/failed on each commit.
Linting, coverage thresholds and enforced code review were not implemented in this phase.
Smoke/Sanity Testing
The CI run acts as a sanity check. A successful build confirms the solution compiles with all references resolving, and the test run confirms the core pipeline still works after each change.

Task 6: Defect Management and Root Cause Analysis
Severity, priority and status change
Category Value Meaning Severity High Produces invalid test results or breaks a core function. Medium A requirement isn’t met or a check is unreliable, however the system still runs correctly. Low Cosmetic, wording or usability issue that doesn’t affect the results Priority High Needs to be fixed before next test run Medium Needs to be Fixed or resolved before final release evaluation. Low Fixed if

time allows it to be, otherwise recorded as a limitation Status
Open Found and recorded but not yet resolved.
Under review Needs a decision, not a code change.
Closed Fixed and verified by re-running the test
Defect lifecycle
The defects discovered, either by automated running of the MSTest tests from Visual Studio's Test Explorer and their summary report output or by our manual evaluation of the GUI, were entered into the defect register. Each defect was cross-referenced to its requirement and test case and given a severity and priority rating. Code defects were fixed and then re-verified by running the complete MSTest suite according to the regression testing in task 3. Requirement-based defects (DEF-01) were not fixed and were escalated as review decisions, since they weren't code errors. Closed defects were checked against the traceability matrix. DEF-03, DEF-04, and DEF-06 were found by reviewing the code and requirements. FR-06 and FR-07 are recorded as scope limitations in Task 2, not defects. NFR-01 results outside Baseline/Baseline (TC-16) are non-conformances reported in Tasks 3, 7 and 8; DEF-01 covers NFR-02. DEF-05 and DEF-06 were fixed in the final code session and verified by two full-suite runs.
Final Defect Register
ID Defect Related requirement and test Severity Priority
Status DEF-01 System sustains about 13FPS on CPU-only YOLOv8n interface against a 15FPS threshold. NFR-02, TC-09 Medium Medium Under review (Task 8 and 10) DEF-02
Parallel test execution leaked the CPU affinity setting between tests, producing inconsistent latency results. NFR-01, NFR-02 High High Closed DEF-03 NFR-03 had defined threshold but no measurement in code NFR-03 Medium High
Closed DEF-04 NFR-08 report check passed unconditionally if the run returned NFR-08, TC-12 to TC-15 High Medium Closed DEF-05 NFR-08 description string cites TC-07, TC-09, and TC-10, which do not match final test case numbering (TC-12 to TC-15.) FR-07, NFR-08 Low Medium Closed
DEF-06 MSTestSettings.cs requests method-level parallel execution while AssemblyInfo.cs sets DoNotParallelize. The effective behaviour has not been confirmed NFR-01, NFR-02 Medium Low
Closed DEF-07 Profile and load pickers start blank, and runs with no selection use Baseline/Baseline without telling the user
NFR-05 Low Low Open DEF-08 Option labels do not explain what each profile or load condition does NFR-05 Low Low
Open Root Cause analysis
DEF-02: CPU affinity leakage between parallel tests. The constrained profile constrains the process to a single core by changing process affinity. The change is then made to the process, not to a specific test. MSTest was running the tests in parallel, and so a constrained test would change the core count for a different running test. Latency results therefore varied depending on what other tests were running in parallel. The root cause was shared process-level state and parallel test execution, which didn't isolate one test from the next. The fix was to add DoNotParallelize and reset the affinity when the test is set up and completed. As a preventative measure, the entire suite is re-run after each change.
DEF-04: NFR-08 check always succeeded. The requirement states that all

defined error conditions are handled without crashing. The report check only verified that the run finished, so it would pass even if an error handler was broken. The root cause was that this requirement was implemented as a runtime flag without individual tests for each error condition. The fix was to add tests for each condition: missing model, corrupted frame, missing input file, and empty batch (TC-12 to TC-15). The report check remains a runtime-only check. As a preventative measure, each NFR is mapped to a test that can fail, as shown in the Task 2 traceability matrix.

DEF-01: FPS too low (below NFR-02). The 15FPS threshold was specified in the Assessment 1 requirements before the system was measured. The test results show that CPU-only YOLOv8n inference reaches only ~13FPS, hence the requirement fails even though the code is correct. The root cause is that the requirement was established without a measured baseline rather than incorrect code. The proposed fix is evaluated in Task 8, with the review in Task 10; As a preventative measure, performance requirement thresholds should be set after establishing a baseline.

How defect analysis improved quality

DEF-03 and DEF-04 mean NFR-03 is now measured and NFR-08 is tested under conditions that can fail. DEF-02 led to sequential testing and more consistent timings. DEF-01 showed a failing requirement can be a requirement problem, which Tasks 8 and 10 take forward, and DEF-07 and DEF-08 produced the Task 7 recommendations.

Task 7: Quality Testing

Area 1: Usability

Risk-based justification

The GUI is the only way a tester configures and runs a scenario, so a confusing interface leads to improper runs and misled results. That would undermine the projects' true purpose. Usability also covers NFR-05, the only requirement that can't be automated, so it needs manual evidence to be verified.

Approach

Manual, scenario-based testing of MainPage against NFR-05. Each test was done on the finished GUI, with screenshots as evidence where applicable.

Test cases

ID	Test	Preconditions	Steps	Expected	Priority	Status
UT-01	Configure and run in ≤3 steps, no code edits (NFR-05)	App launched, MainPage idle	1. Select resource profile 2. Select load condition 3. Click Run	TestRun completes; results header shows the selected profile and load	High	Pass

Configured and run in 3 steps (profile, load condition, Run) with no code edits. Meets NFR-05.

ID	Test	Preconditions	Steps	Expected	Priority	Status
UT-02	Run with no selections made	App freshly launched	Click Run	Test without touching either dropdown	Run starts with sensible defaults, or the user is told what's missing	Medium	Pass with observation

Running with no selections defaulted to Baseline/Baseline. The results header showed the profile, but nothing told the user a default was applied.

ID Test Preconditions Steps Expected Priority Status
UT-03 First-time user completes UT-01 unaided Tester who didn't build the GUI Give the tester only the instruction "run a constrained , high-load test" Completes in ≤3 steps without help High Pass (limited evidence)

Matthew, who didn't build the GUI, found it easy to understand, and completed it in three steps.

ID Test Preconditions Steps Expected Priority Status
UT-04 Results readable and copyable A completed run Select the results text and copy it elsewhere Full summary copies intact Low Pass

The full results summary was readable and copied elsewhere.

ID Test Preconditions Steps Expected Priority Status
UT-05 Unprompted use by an independent tester App launched, MainPage idle, unfamiliar tester, given no instructions, and unaware of being timed Tester explores the GUI and runs a test of their own choosing
Tester configures and starts a run unaided in ≤3 steps Medium
Pass: completed in 3 steps, 7.11s. Tester chose Constrained + High Load; said the option labels didn't make clear what each executes

Findings
NFR-05 is met. The GUI lets a user configure and launch a run in three steps without touching source code, and two other people, Matthew and Devon (an independent tester) found it easy to follow. Two low-severity

observations come out of testing. First (UT-02), the pickers start blank and a run with no selection uses Baseline/Baseline, so a user could think they ran a constrained test when they didn't. Second (UT-05), the option labels don't explain what each profile or load condition does. Both are recorded in the Task 6 defect register as low-severity, low-priority usability defects.

Limitations

Only two people other than the developer tested the GUI. One is a teammate who knows the project and co-wrote the usability cases, and the other a friend with no prior knowledge, so this is supporting evidence and not a usability study. UT-03 has no recorded timing.

No formal usability scale (such as SUS, System Usability Scale) and no accessibility testing.

The GUI only runs the fixed sample images, so real-world workflows aren't covered.

Recommended Improvements

Preselect Baseline in both pickers

Use descriptive labels (e.g. "Constrained: 1 core, +50ms simulated latency")

Show the active configuration next to the Run button

Repeat testing with several unfamiliar testers using timed tasks.

Area 2: Performance

Risk-based justification

The project's central claim is that a system can be functionally correct and still fail real-time requirements. NFR-01 to NFR-03 (latency, frame rate, memory) are the requirements most likely to fail in deployment, so they hold the highest risk and are tested first.

Approach

TC-16 runs a 10-frame batch under every combination of resource profile (Baseline, Constrained) and load condition (Baseline, HighLoad), after a warm-up frame in each cell, and records average per-frame latency, wall-clock throughput, and the NFR-01-03 results. The test asserts that every frame completes. Thresholds are reported but not asserted, because timings vary based on the host machine and would make CI unreliable.

Profile	Load	Avg latency (per frame)	Throughput (frames/wall time)	NFR-01	NFR-02	NFR-03 (peak)
Baseline	Baseline	78.3ms	11.7-13.1FPS	Pass	Fail	Pass (180-186MB)
Baseline	HighLoad	381.6ms	12.5-15.9FPS	Fail	Fail	Pass (214-226MB)
Constrained	Baseline	1773.0ms	0.5-0.6FPS	Fail	Fail	Pass (209-212MB)
Constrained	HighLoad	6078.9ms	0.6-0.7FPS	Fail	Fail	Pass (234-238MB)

Findings

Only the unconstrained, baseline-load case meets NFR-01, and no case meets NFR-02, no frame reached 15FPS in any run, and the closest was 67.2ms against the 66.7ms needed. Memory isn't a concern. Constrained runs are roughly 23x slower per frame and the cause wasn't isolated.

Under HighLoad on the Baseline profile, per-frame latency rose about 4.9x whilst wall-clock throughput reached 15FPS or more in 4 of 6 runs. This is consistent with queuing and not slower processing, though that wasn't isolated either. That throughput comes from a short 10-frame burst and doesn't mean each frame sustains 15FPS. All four cells completed every

frame with none skipped, supporting NFR-04 and NFR-07.
Limitations
Each cell was run six times across sessions, with Baseline/Baseline ranging from 76.0 to 85.1ms. A seventh run recorded 152ms for Baseline/Baseline and was not reproduced or explained. Results come from one machine (Ryzen 7 2700X, 8 core processor, 32GB DDR4 RAM) and two sample images. Memory is the process-wide working set, which isn't reset between cells, so the values aren't independent. Constraints are simulated and not measured on embedded hardware. CI runners will give different figures, so the report uses local results. Detection correctness under HighLoad was checked only by comparing detection counts in a local, uncommitted test, and the detector's thread-safety is not guaranteed.
Recommended Improvements
Repeat on an idle machine, report wall-clock throughput alongside per-frame latency and review the 15FPS and 100ms thresholds against what CPU-only inference can achieve.
Task 8: Quality Metrics and Final Release Decision
Quality metrics
Metric Result Test pass rate (MSTest) 16 of 16 methods pass (100%) Test pass rate (NFR thresholds included) 14 of 16 test cases pass (87.5%). TC-09 and TC-16 fail, 0 blocked, 0 not run
Requirement coverage 20 of 20 requirements have at least one test case Automation coverage 16 of 16 test cases automated. 19 of 20 requirements have an automated test. NFR-05 is verified manually
Requirement results 14 met, 5 partially met (FR-02, FR-06, FR-07, NFR-01, NFR-06), 1 not met (NFR-02) Defects found 8 (DEF-01 to DEF-08) Defect severity 2 High, 3 Medium, 3 Low
Defect status 5 Closed, 2 Open, 1 Under Review Usability testing 5 test cases, none failed, 2 low-severity defects raised
Code coverage Not measured Code quality indicators
CI build and test gate pass. No linting, static analysis or coverage threshold Commit activity Yaacoub: 20
Matthew: 14
Total commits: 34 The gap between the 100% MSTest pass rate and the 87.5% rate that includes NFR thresholds is the projects main finding: the code and tests run correctly while two requirements remain unmet.
Evaluation against requirements and quality attributes
Quality attributeRequirements Result Functional correctness FR-01, FR-03, FR-04, FR-05, FR-08 to FR-11 Met. Detection output, logging, load conditions and error handling work as specified.
Functional correctness (partial)
FR-02, FR-06, FR-07 Partially met. The constraint is CPU-only with no memory cap, and the report has no detection accuracy measure. Detection accuracy (FR-02) is unverified. Performance (latency) NFR-01
Partially met. 78.3ms under Baseline/Baseline only. It fails under Baseline profile with HighLoad (381.6ms), Constrained with Baseline load (1773.0ms) and Constrained with HighLoad (6078.9ms).
Performance (frame rate) NFR-02 Not met. No frame reached 15FPS in any run. Baseline/Baseline averaged 78.3ms per frame (about 12.8FPS). Wall-clock throughput under concurrent load reached 15.1 to 15.9FPS in 4 of 6 runs. Resource use NFR-03 Met. Peak memory was 180 to 238MB against a 512MB limit. Reliability NFR-04, NFR-06, NFR-08 NFR-04, NFR-08: Met (no frames skipped in any

combination, each error condition tested). NFR-06: Partially met (detection counts did not vary across repeated runs; bounding-box variance not tested). Portability across hardware profiles NFR-07 Met under both profiles. Start-up NFR-09 Met. Usability NFR-05 Met. Configured and run in three steps by independent testers, with two low-severity observations. Each requirement's acceptance criterion from Assessment 1 is its test threshold, so the results above are also the acceptance-criteria verdict.

Summary of testing, defects, limitations and residual risks

Testing covered 16 automated test cases, manual usability testing and a performance matrix across all four profile and load combinations. All high-severity defects are closed. Of the three defects not closed, one is Medium (DEF-01, under review) and two are Low.

Results are limited by one test machine, two sample images, six runs per performance cell with one unexplained 152ms outlier, and TC-09 and TC-16 reporting thresholds without asserting them.

Residual risks are:

NFR-02 and NFR-01 (outside Baseline/Baseline) are not met

the resource constraint is CPU-only

detection accuracy is not verified

timing results change with machine load

DEF-07 and DEF-08 remain open

Evaluation of NFR-02

A frame needs 66.7ms or less, and the best observed frame was 67.2ms with an average of 78ms. Measured as throughput under concurrent load it is borderline (12.5 to 15.9FPS). The threshold wasn't lowered to make the test pass. NFR-02 is recorded as not met, and the review of how it should be measured is in Task 10.

Release decision: conditionally ready

The system is conditionally ready for release as a prototype QA tool for measuring non-functional behaviour of an embedded vision workload. It is not ready for release as a real-time embedded deployment, because it does not meet NFR-02 as measured per frame.

The decision rests on the following evidence:

all high-severity defects are closed

every requirement is covered by a test case

reliability, memory, start-up and usability requirements are met

the two unmet NFRs are documented, explained and traceable to DEF-01

The conditions for release are:

The group records NFR-02 as a known limitation and reviews the 15FPS threshold at requirement level (Task 10).

The scope of NFR-01 is stated, applying to the Baseline profile, and constrained and HighLoad results are reported as informational.

DEF-07 and DEF-08 are addressed with the improvements recommended in Task 7.

Task 9: AI-Assisted Development and Critical Reflection

How AI was used

Two tools were used. GitHub Copilot in Visual Studio was used in the IDE for debugging and as a QA reviewer of ProcessFrames, and Claude was used as an assistant for scaffolding (detector, constraint simulator, metrics logger, report generator, GUI), drafting MSTest cases, diagnosing errors and helped plan the CI workflow.

Representative evidence

We keep claude in the loop with everything we've done thus far (Project

with matthew, what we both write in the report, how its shaping out), this is giving us a new test case and making sure that we update the Tasks and keeping up to date.

This shows claude helping us build the CI workflow because we lack the knowledge in CI workflows.
Prompt to Copilot: Review this method for quality risks: error handling, maintainability, and anything that could hide failures

Review, modification and testing
Suggestions weren't taken at face value. Library APIs were checked against decompiled source, code was built and run, and each module got MSTest coverage. A latency anomaly (890-3001ms against ~71-85ms in clean runs) was isolated and reproduced. The first fix (resetting affinity before and after tests) didn't work, so the cause was rehashed, it being parallel tests sharing the process-wide ProcessorAffinity. [assembly: DoNotParallelize] fixed it.
Quality risks
Incorrect logic: The NFR-08 report check passed whenever a run finished, so it couldn't fail (DEF-04, fixed with per-condition tests)
Maintainability: The affinity-reset helper was duplicated across five test classes
Unsuitable assumptions: Thresholds were set before any measurement, and NFR-02 remains under review
Robustness: ProcessFrames catches all exceptions as skipped frames, which could hide failures
Process-wide simulator state (CPU affinity) caused the concurrency bug, and the detector's thread-safety under HighLoad is unverified
Security: The surface is minimal. The real concerns are the model's AGPL licensing and unreviewed third-party packages
Bias: The model is COCO-pretrained and was only tried on two images, so fairness across conditions wasn't checked
Reflection
AI really sped up the process of building the foundations and working with unfamiliar APIs as well as keeping track of what is written in the report itself rather than writing the report. The code output from Claude did set us back only slightly since it based its code on outdated documentation and older versions of the API, but that's where Copilot came in. Given its direct access to code in Visual Studio, as well as up-to-date documentation, the issue was quickly resolved. The temporary thresholds were not an issue considering there was no data available to back the placeholders. Once data was available, they were reconsidered. The concurrency bug was a concern because there would have been a chance that it could have slipped our attention and remained an issue, therefore, producing improper results. If we could do something differently, it would be to rely on GitHub Copilot for in-IDE development and use Claude as a second opinion. Learning how to read library code by decompiling it, analysing tests written by AI, and AI output being inaccurate regardless of what you use helped bring this project to what it is.
Task 10: Quality Improvement and Future Recommendations
Main quality lessons learned
The main lesson learned from the report would seem to be that a pass doesn't necessarily mean a requirement has been met. Both TC-09 and TC-16

do not assert their NFR thresholds, as the absolute timings are machine-dependent. MSTest and CI therefore reported them as passed, but NFR-02 still did not meet the threshold, as the Baseline/Baseline mean was 12.8FPS per frame against the 15FPS required. Throughput from a short concurrent burst reached 15.9FPS in some runs, but NFR-02 is defined per frame. This suggests that the failure may only be visible in the report output, and an observer would miss it if they relied on the pass/fail result.

Another lesson learned concerns the necessity to test both the tests and the code. DEF-04, an NFR-08 check which always passed, and DEF-02, which showed affinity leakage between parallel tests, were inherent to the tests: it seems unlikely a reliable set of test results will be available until the tests are reviewed. Furthermore, DEF-07 and DEF-08 in the GUI were found through exploratory testing, as no scripted test checked for them.

Improvements to the development and QA process

Increasing the visibility of failure of NFRs. Asserting the thresholds on a performance benchmark run (e.g., a fixed machine), so that if an NFR fails, the test is marked as failing rather than just being mentioned in the report.

Tests to be reviewed in PRs as a matter of course. Every test should be proved to fail when the behaviour being measured is broken. If this had been enforced, DEF-04 might have been caught earlier.

Defects remaining open at the time of release (DEF-01, DEF-07, DEF-08) should be fixed based on severity.

Future improvements

Testing suggests an accuracy measure and a labelled data set are needed to satisfy FR-07, as it was only partially met. User-supplied datasets could also be used, along with testing for variance in the bounding boxes. NFR-02 will be reviewed at requirement level: the target should come from the use case, not from the measured result, and a lower target such as 12FPS would be met on average but not by every frame. Until then NFR-02 remains unmet (DEF-01).

Automation can be done by mocking IDetector for testing the pipeline logic without the model file.

CI/CD. It is suggested to add code coverage with a defined threshold, along with linting and static analysis, to the current build and test gate. The summary report from each run can be published as an artifact. Adding a committed test comparing HighLoad and sequential results, including bounding boxes, and considering one detector instance per thread

Monitoring. The latency, FPS and memory metrics from each run can be maintained and compared with previous runs so that a trend indicating regression can be identified.

Security. The NuGet dependencies can be scanned for known vulnerabilities, and the model file can be checked against a known checksum before loading.

Usability and accessibility. When no option is selected a message can be displayed (DEF-07), a short explanation can be added to the option labels (DEF-08), and keyboard navigation, screen reader labels and contrast tests can be included in manual tests.

Evolving toward continuous quality engineering

Beyond the current build-and-test gate, a fast pipeline could run alongside slower performance tests before each release. Release gates

would include no open critical defects, NFR thresholds asserted on a fixed benchmark machine, and tracked FPS, latency and defect trends across runs. Turning DEF-04 into a standing rule that every NFR check must be able to fail is the main practice carried forward.

Task 11: Team Contribution and Professional Reflection

Contribution Statements

Yaacoub: I built the detector, pipeline runner, resource constraint simulator, metrics logger, summary report generator, load conditions, error handling and the MAUI GUI; set up the repo, branches and GitHub Actions workflow; wrote the memory check and TC-16 performance matrix. Me and Matthew wrote usability test cases. I created the new word document and maintained the structure of it throughout.

Matthew: I wrote Tasks 2, 4, 6, 8, and 10. I also made the automated test cases for NFR-02, NFR-04, NFR-06, and NFR-07, and an MSTest case for each error condition under NFR-08 and wrote the GUI tests together with Yaacoub.

Evidence

Commit activity

42 commits total

Yaacoub: 24

Matthew: 18

Individual Reflection

Yaacoub: All in all, this was genuinely one of my favourite projects to work on. Not only do I get to work on a GUI and Test Cases, but I also get to work on Computer Vision/Firmware. Matthew and I communicated constantly throughout the project, giving each other status updates and new iteration updates to make sure we stayed on track, and by splitting up test cases amongst each other, peer review was needed to ensure a high standard. This was a lengthy project, but it was very much worth it considering the impact that it could have on the future of Computer Vision. I learnt a lot about AI-assisted coding and MSTests, making Software Quality Assurance a key skill of mine, which will be useful in the future.

Matthew: I enjoyed my time working on this project. I helped Yaacoub make the tests Assisted by AI and helped in making the Dataflow/Architecture diagram. The making of the tests being assisted with AI really help me learned a lot on what I should include in my prompts to ensure that the tests won't be defective or not working at all. This project really did help me learn the key skill of Software Quality Assurance which would prove to be useful for my future.

References

Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), Usability evaluation in industry (pp. 189-194). Taylor & Francis. https://www.researchgate.net/publication/319394819_SUS_--_a_quick_and_dirty_usability_scale

Chung, L., & do Prado Leite, J. C. S. (2009). On non-functional requirements in software engineering. In A. T. Borgida, V. K. Chaudhri, P. Giorgini, & E. S. Yu (Eds.), Conceptual modeling: Foundations and applications (Lecture Notes in Computer Science, Vol. 5600, pp. 363-379). Springer. https://doi.org/10.1007/978-3-642-02463-4_19

Chung, L., Nixon, B. A., & Yu, E. (1995). Using non-functional

requirements to systematically support change. In Proceedings of the 1995 IEEE International Symposium on Requirements Engineering (RE'95) (pp. 132–139). IEEE. https://doi.org/10.1109/ISRE.1995.512554

IEEE. (2010). IEEE standard classification for software anomalies (IEEE Std 1044-2009). https://doi.org/10.1109/IEEESTD.2010.5399061

ISO/IEC. (2011). Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models (ISO/IEC 25010:2011). https://iso25000.com/index.php/en/iso-25000-standards/iso-25010

ISO/IEC/IEEE. (2021). Software and systems engineering — Software testing — Part 3: Test documentation (ISO/IEC/IEEE 29119-3:2021). https://ieeexplore.ieee.org/document/9687477

Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '94) (pp. 152–158). ACM. https://doi.org/10.1145/191666.191729